UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

S4

Búsqueda ciega: BFS y DFS

Actividad Parcial 2 · Fundamentos de Inteligencia Artificial

Tema 2.4 · Solución de laberintos y comparación de algoritmos

Alumno	Rogerio Aguilar Bello
Matrícula	24305118
Grupo / cuatrimestre	171 · 7.º
Docente	Mtro. Javier García Lara
Tipo de entrega	Individual · 50 pts
Fecha de entrega	25 de octubre de 2026
Referencias del tema	Bohm (2021), cap. 3
Repositorio GitHub	https://github.com/RogerNika/Rogerio-Aguilar-Bello-FIA

Plus de esta actividad: animación paso a paso (GIF) que muestra lado a lado cómo exploran BFS y DFS, con cuadros incluidos en el documento.

Capítulo 1. Planteamiento del problema

1.1 Contexto

Los algoritmos de búsqueda ciega (no informada) exploran el espacio de estados sin ninguna pista sobre qué tan cerca está la meta. Son la base de la navegación de robots, la generación de niveles de videojuegos y el enrutamiento en circuitos. Los dos más conocidos son la búsqueda en anchura (BFS) y la búsqueda en profundidad (DFS).

1.2 Problema

Dado un laberinto en cuadrícula, encontrar un camino desde la entrada (esquina superior izquierda) hasta la salida (esquina inferior derecha) y comparar BFS y DFS en **nodos expandidos**, **memoria** (tamaño máximo de la frontera), **longitud del camino** y **tiempo**.

1.3 Datos del problema

Laberintos	Generados con retroceso recursivo y semilla fija (reproducibles). Tamaños 15×15, 31×31, 51×51 y 71×71.
Tipos	**Perfecto**: un único camino entre dos celdas. **Con ciclos**: se abre 10 % de muros extra para crear caminos alternativos.
Experimento	10 semillas por tamaño y tipo (promedios); inicio en (1, 1) y meta en (n − 2, n − 2).
Estado / acciones	Celda (fila, col); mover arriba, derecha, abajo o izquierda si la celda es libre; costo 1 por paso.

Capítulo 2. Objetivos

2.1 Objetivo general

Comparar BFS y DFS mediante su implementación en Python sobre laberintos de distintos tamaños y estructuras para determinar en qué condiciones conviene cada uno.

2.2 Objetivos específicos

- **Representar** el laberinto como grafo de celdas **definiendo** nodos, aristas y vecindad **para** aplicar algoritmos de búsqueda sobre él.
- **Implementar** BFS (cola) y DFS (pila) con el mismo orden de vecinos **para** que la comparación sea justa.
- **Medir** nodos expandidos, frontera máxima, longitud del camino y tiempo **mediante** 10 semillas por configuración **para** sustentar las conclusiones con datos.
- **Analizar** la complejidad temporal y espacial teórica frente a la observada **contrastando** ambos resultados **para** concluir qué algoritmo es más idóneo según el problema.

Capítulo 3. Fundamentos

Representación. Cada celda libre es un nodo y dos celdas adyacentes libres forman una arista; se guarda una lista de adyacencia. El factor de ramificación b es a lo sumo 4 (3 al desplazarse sin volver atrás).

BFS expande los nodos por niveles con una cola FIFO. Es completo y, con costo uniforme por paso, **óptimo** (encuentra el camino más corto); su tiempo y espacio son $O(b^d)$ en búsqueda en árbol, y $O(V + E)$ con control de visitados (Russell y Norvig, 2020; Cormen et al., 2022). **DFS** profundiza por una rama con una pila LIFO antes de

retroceder; **no garantiza** el camino más corto. Su espacio teórico es O(b·m), con m la profundidad máxima, y en grafos finitos con visitados es completo.

Decisión de implementación. BFS marca un nodo como visitado al *encolarlo*; DFS lo marca al *desapilarlo*, de modo que un mismo nodo puede estar varias veces en la pila (afecta la frontera medida).

Capítulo 4. Metodología y procedimiento

- Generar el laberinto (perfecto o con ciclos) con una semilla fija.
- Construir la vecindad de cada celda y ejecutar BFS y DFS desde la entrada hasta la meta.
- Registrar nodos expandidos, tamaño máximo de la frontera, longitud del camino y tiempo (mínimo de 3 repeticiones).
- Promediar 10 semillas por tamaño y tipo, y comparar con la teoría.
- Generar la animación paso a paso para visualizar la diferencia (plus).

Capítulo 5. Desarrollo: diseño de la solución

Grafo	Nodos = celdas libres; aristas = celdas adyacentes libres; sin pesos (costo 1).
BFS	deque como frontera; padres para reconstruir el camino.
DFS	Lista como pila; vecinos apilados en orden inverso para explorar primero «arriba».
Meta	Se detiene al expandir la celda de salida.
Complejidad esperada	Ambos O(V + E) con visitados; BFS garantiza camino mínimo y DFS no.

Capítulo 6. Resultados

Tipo	n	Algoritmo	Celdas libres	Expandidos	Frontera máx.	Camino (pasos)	Tiempo (ms)
perfecto	15×15	BFS	97	65	3	49.2	0.13
perfecto	15×15	DFS	97	64	3	49.2	0.15
perfecto	31×31	BFS	449	253	4	176.8	0.39
perfecto	31×31	DFS	449	293	6	176.8	0.53
perfecto	51×51	BFS	1249	774	6	448.8	1.22
perfecto	51×51	DFS	1249	809	14	448.8	1.47
perfecto	71×71	BFS	2449	1615	7	890.8	2.55
perfecto	71×71	DFS	2449	1486	22	890.8	2.74
con ciclos	15×15	BFS	103	85	7	30.2	0.13
con ciclos	15×15	DFS	103	45	7	38.0	0.08
con ciclos	31×31	BFS	480	446	15	71.6	0.69
con ciclos	31×31	DFS	480	163	21	120.4	0.29
con ciclos	51×51	BFS	1342	1304	26	115.4	2.07
con ciclos	51×51	DFS	1342	460	55	270.2	0.85
con ciclos	71×71	BFS	2621	2554	38	177.2	4.07
con ciclos	71×71	DFS	2621	635	79	452.6	1.16

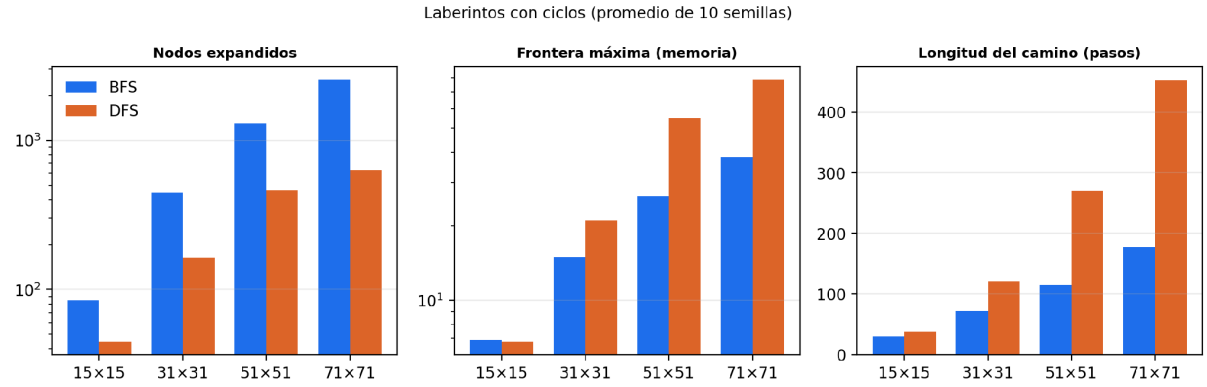


Figura 1. Nodos expandidos, frontera máxima y longitud del camino en laberintos con ciclos (escala logarítmica en los dos primeros).

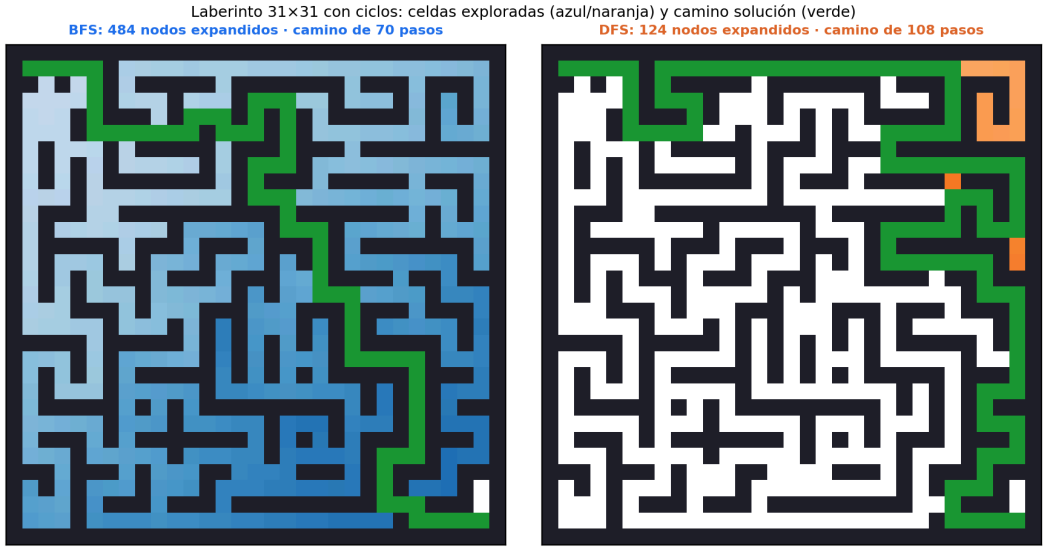


Figura 2. Laberinto 31×31 con ciclos: BFS expandió 484 nodos y halló un camino de 70 pasos; DFS expandió 124 y halló uno de 108 pasos.

Capítulo 7. Código Python

Archivo `fia_s4_bfs_dfs.py` (requiere numpy, matplotlib y pillow).

```
"""
FIA - Actividad Parcial 2 - Sección 4 (2.4 Búsqueda ciega: BFS / DFS)
Resuelve laberintos en cuadrícula con Búsqueda en Anchura (BFS) y en Profundidad (DFS) y los compara.

  Grafo      : cada celda libre es un nodo; aristas entre celdas libres adyacentes (lista de adyacencia)
  Estado     : celda (fila, col)      Acciones: arriba, derecha, abajo, izquierda      Costo: 1 por paso
  Inicio/Meta: esquina superior izquierda / inferior derecha
  Métricas   : nodos expandidos, tamaño máximo de la frontera (memoria), longitud del camino, tiempo

Requisitos: pip install numpy matplotlib pillow
Uso:        python fia_s4_bfs_dfs.py   (genera figuras, animacion_bfs_dfs.gif y resultados_s4.json)
"""
import json, random, time
from collections import deque
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image, ImageDraw

ORDEN = [(-1, 0), (0, 1), (1, 0), (0, -1)]  # arriba, derecha, abajo, izquierda (orden fijo de expansión)
```

```
# ---------------------------------------------------------------------------- laberinto
def generar_laberinto(n, semilla, prob_ciclos=0.0):
    """n impar. 1 = muro, 0 = libre. Retroceso recursivo (iterativo) => laberinto perfecto (camino único).
    prob_ciclos > 0 abre muros extra y crea caminos alternativos ("laberinto con ciclos")."""
    rnd = random.Random(semilla); g = np.ones((n, n), dtype=np.uint8); g[1, 1] = 0; pila = [(1, 1)]
    while pila:
        r, c = pila[-1]
        vec = [(r + 2 * dr, c + 2 * dc, dr, dc) for dr, dc in ORDEN if 0 < r + 2 * dr < n - 1 and 0 < c + 2 * dc <
n - 1 and g[r + 2 * dr, c + 2 * dc]]
        if vec:
            nr, nc, dr, dc = rnd.choice(vec); g[r + dr, c + dc] = 0; g[nr, nc] = 0; pila.append((nr, nc))
        else:
            pila.pop()
    if prob_ciclos > 0:
        for r in range(1, n - 1):
            for c in range(1, n - 1):
                if g[r, c] and rnd.random() < prob_ciclos and ((g[r - 1, c] == 0 and g[r + 1, c] == 0) or (g[r, c -
1] == 0 and g[r, c + 1] == 0)):
                    g[r, c] = 0
    return g


def vecinos(g, nodo):
    r, c = nodo; n = g.shape[0]
    for dr, dc in ORDEN:
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < n and g[nr, nc] == 0:
            yield (nr, nc)


def camino(padres, meta):
    p, nodo = [], meta
    while nodo is not None:
        p.append(nodo); nodo = padres[nodo]
    return p[::-1]


# ---------------------------------------------------------------------------- algoritmos
def bfs(g, ini, meta, registrar=False):
    """BFS: cola FIFO. Marca visitado al ENCOLAR. Óptimo en número de pasos (costo uniforme)."""
    t0 = time.perf_counter(); frontera = deque([ini]); padres = {ini: None}; orden = []; max_f = 1
    while frontera:
        max_f = max(max_f, len(frontera)); nodo = frontera.popleft(); orden.append(nodo)
        if nodo == meta:
            break
        for v in vecinos(g, nodo):
            if v not in padres:
                padres[v] = nodo; frontera.append(v)
    ok = meta in padres and orden[-1] == meta
    return dict(ok=ok, camino=camino(padres, meta) if ok else None, expandidos=len(orden), max_frontera=max_f,
                tiempo=time.perf_counter() - t0, orden=orden if registrar else None)


def dfs(g, ini, meta, registrar=False):
    """DFS: pila LIFO. Marca visitado al DESAPILAR. No garantiza el camino más corto."""
    t0 = time.perf_counter(); pila = [(ini, None)]; padres = {}; orden = []; max_f = 1
    while pila:
        max_f = max(max_f, len(pila)); nodo, padre = pila.pop()
        if nodo in padres:
            continue
        padres[nodo] = padre; orden.append(nodo)
        if nodo == meta:
            break
        for v in reversed(list(vecinos(g, nodo))):   # reversed => se explora primero "arriba"
            if v not in padres:
                pila.append((v, nodo))
    ok = meta in padres
    return dict(ok=ok, camino=camino(padres, meta) if ok else None, expandidos=len(orden), max_frontera=max_f,
                tiempo=time.perf_counter() - t0, orden=orden if registrar else None)


# ---------------------------------------------------------------------------- experimentos
def experimento(tamanos=(15, 31, 51, 71), semillas=range(10)):
    filas = []
    for tipo, pc in (("perfecto", 0.0), ("con ciclos", 0.10)):
        for n in tamanos:
```

```
            acum = {"BFS": [], "DFS": []}
            for s in semillas:
                g = generar_laberinto(n, s, pc); ini, meta = (1, 1), (n - 2, n - 2)
                for nombre, fn in (("BFS", bfs), ("DFS", dfs)):
                    r = fn(g, ini, meta); assert r["ok"]
                    r["tiempo"] = min(fn(g, ini, meta)["tiempo"] for _ in range(3))
                    acum[nombre].append(r)
            for nombre in acum:
                R = acum[nombre]
                filas.append(dict(tipo=tipo, n=n, algoritmo=nombre, libres=int(np.mean([(generar_laberinto(n, s,
pc) == 0).sum() for s in semillas])),
                                  expandidos=float(np.mean([r["expandidos"] for r in R])),
max_frontera=float(np.mean([r["max_frontera"] for r in R])),
                                  largo=float(np.mean([len(r["camino"]) - 1 for r in R])),
tiempo_ms=float(np.mean([r["tiempo"] for r in R]) * 1000)))
    return filas


# ---------------------------------------------------------------------------- gráficas
COL = {"BFS": "#1F6FEB", "DFS": "#E0662B"}


def fig_barras(filas, ruta="s4_metricas.png"):
    fig, axs = plt.subplots(1, 3, figsize=(11, 3.6), dpi=200)
    ncon = [f for f in filas if f["tipo"] == "con ciclos"]; ns = sorted({f["n"] for f in ncon}); x =
np.arange(len(ns))
    for ax, key, tit in zip(axs, ("expandidos", "max_frontera", "largo"), ("Nodos expandidos", "Frontera máxima
(memoria)", "Longitud del camino (pasos)")):
        for i, alg in enumerate(("BFS", "DFS")):
            ax.bar(x + (i - .5) * .38, [next(f[key] for f in ncon if f["n"] == n and f["algoritmo"] == alg) for n
in ns], .38, color=COL[alg], label=alg)
        ax.set_xticks(x, [f"{n}×{n}" for n in ns]); ax.set_title(tit, fontsize=9, fontweight="bold");
ax.grid(alpha=.25, axis="y")
        if key != "largo": ax.set_yscale("log")
    axs[0].legend(frameon=False); fig.suptitle("Laberintos con ciclos (promedio de 10 semillas)", fontsize=10)
    fig.tight_layout(); fig.savefig(ruta, bbox_inches="tight"); plt.close(fig)


def pintar(g, visitados, cam=None, ini=None, meta=None, esc=1):
    n = g.shape[0]; img = np.full((n, n, 3), 255, np.uint8); img[g == 1] = (30, 30, 40)
    for k, (r, c) in enumerate(visitados):
        img[r, c] = (200, 215, 245)
    return img


def fig_solucion(g, rb, rd, ruta="s4_soluciones.png"):
    fig, axs = plt.subplots(1, 2, figsize=(9, 4.7), dpi=200)
    for ax, r, alg in zip(axs, (rb, rd), ("BFS", "DFS")):
        n = g.shape[0]; img = np.full((n, n, 3), 1.0, float); img[g == 1] = (.12, .12, .16)
        orden = r["orden"]; cmap = plt.get_cmap("Blues" if alg == "BFS" else "Oranges")
        for k, (a, b) in enumerate(orden): img[a, b] = cmap(.25 + .5 * k / len(orden))[:3]
        for a, b in r["camino"]: img[a, b] = (.1, .6, .2)
        ax.imshow(img); ax.set_xticks([]); ax.set_yticks([])
        ax.set_title(f"{alg}: {r['expandidos']} nodos expandidos · camino de {len(r['camino']) - 1} pasos",
fontsize=8, fontweight="bold", color=COL[alg])
    fig.suptitle("Laberinto 31×31 con ciclos: celdas exploradas (azul/naranja) y camino solución (verde)",
fontsize=9)
    fig.tight_layout(); fig.savefig(ruta, bbox_inches="tight"); plt.close(fig)


# ---------------------------------------------------------------------------- PLUS: animación
def render(g, orden, k, cam, color, escala=12):
    n = g.shape[0]; img = np.full((n, n, 3), 255, np.uint8); img[g == 1] = (30, 30, 40)
    for (a, b) in orden[:k]: img[a, b] = color
    if k >= len(orden) and cam:
        for (a, b) in cam: img[a, b] = (30, 170, 70)
    elif k > 0:
        a, b = orden[k - 1]; img[a, b] = (220, 30, 30)
    img[1, 1] = (0, 0, 0) if g[1, 1] == 0 else img[1, 1]
    return Image.fromarray(img).resize((n * escala, n * escala), Image.NEAREST)


def animacion(g, rb, rd, ruta="animacion_bfs_dfs.gif", pasos=70):
    n = g.shape[0]; E = 12; frames = []; maxk = max(len(rb["orden"]), len(rd["orden"]))
    ks = sorted(set(int(round(i * maxk / pasos)) for i in range(pasos + 1))) + [maxk] * 8
    for k in ks:
        kb, kd = min(k, len(rb["orden"])), min(k, len(rd["orden"]))
```

```
        a = render(g, rb["orden"], kb, rb["camino"], (120, 160, 235), E); b = render(g, rd["orden"], kd,
rd["camino"], (240, 160, 110), E)
        canvas = Image.new("RGB", (2 * n * E + 12, n * E + 26), "white"); canvas.paste(a, (0, 26)); canvas.paste(b,
(n * E + 12, 26))
        d = ImageDraw.Draw(canvas); d.text((6, 6), f"BFS  expandidos: {kb}", fill=(31, 111, 235)); d.text((n * E +
18, 6), f"DFS  expandidos: {kd}", fill=(224, 102, 43))
        frames.append(canvas)
    frames[0].save(ruta, save_all=True, append_images=frames[1:], duration=90, loop=0)
    return frames


def tira(g, rb, rd, ruta="s4_tira_animacion.png", cortes=(0.1, 0.35, 0.7, 1.0)):
    fig, axs = plt.subplots(2, len(cortes), figsize=(9.2, 4.9), dpi=200)
    for j, q in enumerate(cortes):
        for i, (r, col, alg) in enumerate(((rb, (120, 160, 235), "BFS"), (rd, (240, 160, 110), "DFS"))):
            k = max(1, int(len(r["orden"]) * q)); im = render(g, r["orden"], k, r["camino"], col, 8)
            axs[i, j].imshow(im); axs[i, j].set_xticks([]); axs[i, j].set_yticks([])
            axs[i, j].set_title(f"{alg}: {k} nodos", fontsize=7, color=COL[alg], fontweight="bold")
    fig.suptitle("Plus: la exploración paso a paso (cuadros de la animación)", fontsize=9); fig.tight_layout()
    fig.savefig(ruta, bbox_inches="tight"); plt.close(fig)


if __name__ == "__main__":
    filas = experimento()
    print(f"{'tipo':11s} {'n':>3s} {'alg':4s} {'libres':>6s} {'expand.':>9s} {'frontera':>9s} {'largo':>7s}
{'t(ms)':>8s}")
    for f in filas:
        print(f"{f['tipo']:11s} {f['n']:>3d} {f['algoritmo']:4s} {f['libres']:>6d} {f['expandidos']:>9.0f}
{f['max_frontera']:>9.0f} {f['largo']:>7.1f} {f['tiempo_ms']:>8.2f}")
    g = generar_laberinto(31, 3, 0.10); rb = bfs(g, (1, 1), (29, 29), True); rd = dfs(g, (1, 1), (29, 29), True)
    print(f"\nEjemplo 31x31 con ciclos: BFS exp={rb['expandidos']} camino={len(rb['camino'])-1} | DFS
exp={rd['expandidos']} camino={len(rd['camino'])-1}")
    fig_barras(filas); fig_solucion(g, rb, rd)
    g2 = generar_laberinto(21, 5, 0.10); b2 = bfs(g2, (1, 1), (19, 19), True); d2 = dfs(g2, (1, 1), (19, 19), True)
    animacion(g2, b2, d2); tira(g2, b2, d2)
    json.dump(dict(filas=filas, ejemplo=dict(bfs_exp=rb["expandidos"], dfs_exp=rd["expandidos"],
bfs_largo=len(rb["camino"]) - 1, dfs_largo=len(rd["camino"]) - 1)),
              open("resultados_s4.json", "w"), indent=1)
```

Salida real de la ejecución:

```
tipo          n alg  libres   expand.  frontera   largo    t(ms)
perfecto     15 BFS      97        65         3    49.2     0.13
perfecto     15 DFS      97        64         3    49.2     0.15
perfecto     31 BFS     449       253         4   176.8     0.39
perfecto     31 DFS     449       293         6   176.8     0.53
perfecto     51 BFS    1249       774         6   448.8     1.22
perfecto     51 DFS    1249       809        14   448.8     1.47
perfecto     71 BFS    2449      1615         7   890.8     2.55
perfecto     71 DFS    2449      1486        22   890.8     2.74
con ciclos   15 BFS     103        85         7    30.2     0.13
con ciclos   15 DFS     103        45         7    38.0     0.08
con ciclos   31 BFS     480       446        15    71.6     0.69
con ciclos   31 DFS     480       163        21   120.4     0.29
con ciclos   51 BFS    1342      1304        26   115.4     2.07
con ciclos   51 DFS    1342       460        55   270.2     0.85
con ciclos   71 BFS    2621      2554        38   177.2     4.07
con ciclos   71 DFS    2621       635        79   452.6     1.16

Ejemplo 31x31 con ciclos: BFS exp=484 camino=70 | DFS exp=124 camino=108
```

Capítulo 8. Análisis y discusión

8.1 Laberintos perfectos

Con un único camino posible, ambos algoritmos devuelven la misma ruta (por ejemplo 891 pasos en 71×71) y los nodos expandidos son parecidos (BFS 1615, DFS 1486); la elección de algoritmo prácticamente no cambia la calidad de la solución.

8.2 Laberintos con ciclos

Aparece el compromiso clásico. BFS devuelve siempre el camino más corto (177 pasos en 71×71) pero expande 2554 nodos, casi toda la zona libre (2621 celdas). DFS expande solo 635 nodos (4.0 veces menos) y es más rápido (1.2 frente a 4.1 ms), pero su camino es 2.6 veces más largo (453 pasos).

8.3 Memoria: la teoría y la medición

La teoría sugiere que DFS usa menos memoria ($O(b \cdot m)$) que BFS ($O(b^d)$). En estos laberintos de corredores estrechos se observó lo contrario: la frontera máxima de BFS fue de 38 y la de DFS de 79 (71×71 con ciclos). La razón es que la frontera de BFS es el «anillo» de celdas a igual distancia, que en corredores es angosto, mientras que la pila de DFS conserva pendientes en cada bifurcación a lo largo de un camino muy largo y, al marcar visitados al desapilar, puede guardar nodos duplicados. La ventaja de memoria de DFS depende de que el árbol sea ancho y la solución profunda, y de usar marcado al apilar.

8.4 Cuándo conviene cada uno

- **BFS** cuando se necesita el camino más corto o la solución está cerca de la entrada.
- **DFS** cuando basta *cualquier* solución y se tolera un camino largo, o cuando se quiere explorar todo el espacio (por ejemplo, para generar laberintos).
- Ninguno usa información de la meta; en la Actividad 5 se compara con A*, que sí la usa.

8.5 Limitaciones

- El resultado de DFS depende del orden de vecinos y de la posición de la meta (esquina opuesta); otro orden cambiaría los nodos expandidos.
- Los tiempos corresponden a una implementación en Python en un solo equipo y son aproximados.
- Los laberintos son sintéticos; en un mapa real los costos por paso no serían uniformes.

Capítulo 9. Conclusiones

- BFS garantizó el camino más corto en todos los casos; en laberintos con ciclos DFS encontró caminos hasta 2.6 veces más largos.
- DFS expandió 4.0 veces menos nodos en 71×71 con ciclos, pero esa ventaja de velocidad se paga con calidad de la solución.
- La memoria depende de la estructura del problema y de la implementación; conviene medirla y no asumirla solo por la teoría.
- Representar el problema como grafo y registrar métricas permite elegir el algoritmo con evidencia.

Capítulo 10. Plus: animación paso a paso de BFS y DFS

Se generó `animacion_bfs_dfs.gif`, que muestra lado a lado cómo exploran BFS (azul) y DFS (naranja) un laberinto 21×21 con ciclos, con el camino solución en verde. BFS avanza como una onda circular; DFS se interna por un solo corredor hasta agotarlo. La figura siguiente reúne cuatro cuadros de esa animación.

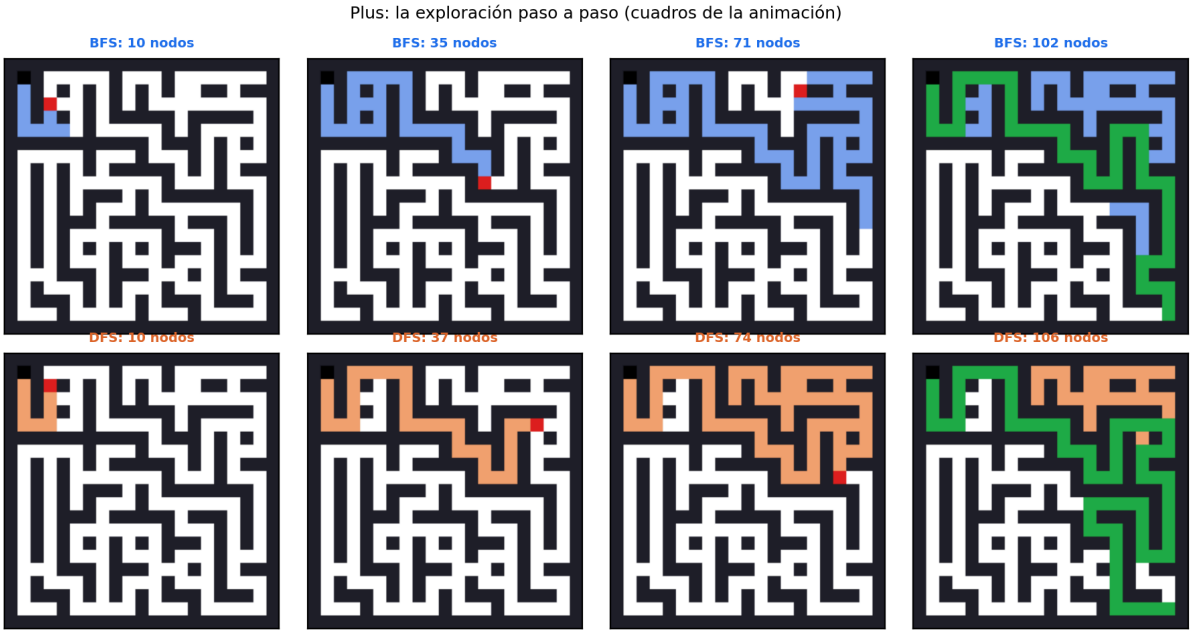


Figura 3. Cuadros de la animación a distintos porcentajes de avance (arriba BFS, abajo DFS). La animación completa está en el archivo GIF del repositorio.

Capítulo 11. Referencias (APA 7.ª ed.)

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Anexo A. Lista de cotejo (S4)

#	Criterio de evaluación	S4
1	Portada, tema, sección y datos del problema completos en el PDF	✓
2	Planteamiento, formulación y procedimiento claramente explicados	✓
3	Código Python funcional y comentado, probado con datos de entrada	✓
4	Solución enmarcada, con gráficos, tablas o diagramas de resultados	✓
5	Repositorio GitHub con carpeta del alumno, README y PDFs	Pendiente: subir a GitHub
6	Declaración de IA utilizada, referencias APA 7 y conclusiones reflexivas	✓

Declaración de uso de IA

Herramienta: Claude (Anthropic), asistente conversacional.
Prompt: De este documento verifica y genera una animacion para la valoracion deltrabajo y dale diseño al formato
Uso: apoyo para diseñar el experimento, escribir el código de BFS y DFS, generar la animación y redactar el borrador.
Verificación: el código se ejecutó y su salida se incluye en el capítulo 7; las cifras del texto se calcularon desde el archivo de resultados. Revisión personal por el alumno: [fecha].